# Software Requirements Specification

**for**

# Rail and Road-based Supply Chain Distribution System

**Version 2.1**

**Prepared by Group 24**

**Members:**

240688E - Vithanage K. O.

240347J - Kawya W. W. D.

240617M - Shameera M. N.F.

240212R - Haneef H.M.

240005K - Aathiththan K.

**Department of Computer Science and Engineering**
**University of Moratuwa**

**11th October 2026**

*Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.*

# Table of Contents

**1. Introduction** **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
**2. Overall Description** **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 3
2.4 Operating Environment 3
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 4
**3. External Interface Requirements** **5**
3.1 User Interfaces 5
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
**4. System Features** **7**
4.1 Train Cargo Capacity & Space Consumption Scheduling 7
4.2 Order Placement & Route Matching 8
4.3 Last-Mile Delivery & Truck Scheduling 9
4.4 Driver & Assistant Roster & Constraint Management 10
4.5 Management Reporting & Analytics 11
4.6 Database Integrity & ACID Enforcement (System Internal Feature) 11
**5. Other Nonfunctional Requirements** **12**
5.1 Performance Requirements 12
5.2 Safety Requirements 13
5.3 Security Requirements 13
5.4 Software Quality Attributes 14
5.5 Business Rules 14
**6. Other Requirements** **15**
**Appendix A: Glossary** **15**
**Appendix B: Analysis Models** **16**
**Appendix C: Data Dictionary / Entity Tables** **17**
**Appendix D: To Be Determined List** **27**
**Appendix E: Changes from Version 1.0** **27**

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| Group 24 | 25/07/2026 | Initial submission covering sections 1-6 and appendices | 1.0 |
| Group 24 | 10/10/2026 | Revised after the implementation review: customer delivery areas, train destination store, stored requested delivery date, truck trip rules enforced on create and edit, customer portal security, corrected data dictionary and ER diagram. All changes are listed in Appendix E. | 2.0 |
| Group 24 | 11/10/2026 | Aligned with the system as built and tested: delivery rule procedure and shipment weekday triggers added to the program units, route distance and fleet report definitions clarified, test timetable stated as an assumption, module specifications referenced. These changes are listed at the end of Appendix E. | 2.1 |

# 1. Introduction

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to specify the software and database requirements for Version 2.0 of the **Kandypack Rail and Road-based Supply Chain Distribution System**. This document details the database-driven logistics platform designed to replace legacy Excel-based tracking systems, covering bulk rail transportation, last-mile truck dispatching, roster management, a customer self-service portal, and managerial reporting.

Version 2.0 revises Version 1.0 (25 July 2026) to reflect the design decisions made while the system was implemented and reviewed. Requirements added in this version are marked **(v2)**, and every change is listed in Appendix E. Version 2.1 (11 October 2026) aligns the document with the system as built and tested; those changes are listed at the end of Appendix E.

## 1.2 Document Conventions

**Priority Levels:** Functional requirements are marked with priorities (**High**, **Medium**, **Low**). Higher-level requirements pass their priority to child requirements unless explicitly noted.

**Database Objects:** Names of database tables, columns, procedures, functions, and triggers (e.g., `TruckTrip`, `RequestedDeliveryDate`, `sp_place_order`) are written in monospaced font.

**Business Constraints:** Key constraints (e.g., mandatory lead times, weekly hour limits) are explicitly highlighted in **bold**.

**Version Markers:** Requirements introduced in Version 2.0 carry the marker **(v2)** after their identifier.

## 1.3 Intended Audience and Reading Suggestions

This document is written for:

- **Database Developers & Engineers:** To guide schema design, primary/foreign key definitions, stored procedures, triggers, and indexing strategies.
- **Software Developers:** To understand REST API and JDBC data interaction requirements for backend and web interface integration.
- **Quality Assurance (QA) Testers:** To formulate test cases validating ACID transactions, capacity overflow, roster constraint rules, and access control.
- **Project Managers & Course Evaluators:** To verify system scope, deliverables, and domain modelling alignment. Appendix E summarises what changed since Version 1.0.

## 1.4 Product Scope

The Kandypack Logistics Platform is a database-driven distribution system designed for Kandypack, an FMCG manufacturing company based in Kandy, Sri Lanka. The platform

handles bulk rail distribution between Kandy and six major destination cities (namely: Colombo, Negombo, Galle, Matara, Jaffna and Trincomalee), last-mile truck delivery from each city's store, and a portal through which wholesale and retail customers place and track their own orders. It replaces existing manual Excel tracking to resolve delivery inefficiencies caused by growing order volumes.

### 1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Kandypack Supply Chain & Logistics Operations Specifications (Project 3 Brief).
- Sri Lanka Railways Fixed Cargo & Route Schedule Guidelines.
- Group 24, Software Requirements Specification, Version 1.0, 25 July 2026.
- Group 24, Module Technical Specifications (Rail Cargo Allocation; Roster, Truck Trip and Delivery; Customer Order and Portal), project repository folder docs/Project-Documentation, October 2026.

## 2. Overall Description

### 2.1 Product Perspective

The system is a new, self-contained product. Kandypack currently uses a manual Excel-based tracking system, which this platform replaces entirely; it is not a follow-on version of any existing software. It is a web-based platform: a relational MySQL database (accessed via JDBC) sits behind a Spring Boot service layer, which exposes REST APIs to a React web interface. Internal staff use the main interface; customers use a separate customer portal within the same application.

### 2.2 Product Functions

The functions provided by this system, at a high level, are to:

- **Customer & Order Management:**
    - Register customers and link each one to the delivery area in which their address lies
    - Create and validate orders, assigning each to the delivery route that covers the customer's delivery area
    - Register orders containing multiple line items, each with a specific space consumption rate, and store the requested delivery date
- **Customer Self-Service (v2):**
    - Let customers register, log in, maintain their profile, place orders, and track their own orders and deliveries
- **Rail Cargo Scheduling & Overflow:**
    - Schedule order items onto train runs bound for the order's destination store, checking the capacity of each run and sending any overflow to the next departure

- **Store & Last-Mile Delivery Operations:**
  - Keep records of stores, their delivery areas and routes, and the goods they hold
  - Assign trucks from the route's own store, drivers, and assistants to scheduled routes
- **Driver & Assistant Roster Management:**
  - Enforce driver and assistant rostering rules (weekly working-hour limits and consecutive-trip constraints) whenever a trip is created or changed
- **Delivery Tracking & Reporting:**
  - Track and update the status of each delivery, keeping the order status in step
  - Generate the management reports under Section 4.5, on screen and as PDF

## 2.3 User Classes and Characteristics

These are the user classes expected. Each staff class corresponds to a system role, shown in brackets.

- **Order Clerks** (`ORDER_MANAGER`): create and manage customer orders and customer records; frequently used, moderate technical expertise
- **Logistics / Station Managers** (`RAIL_DISPATCHER`, `FLEET_MANAGER`): personnel at headquarters and destination station stores overseeing train loading and unloading, truck/route assignment, and the fleet; frequently used, central to last-mile operations
- **Roster / Fleet Coordinators** (`ROSTER_DISPATCHER`): staff managing driver and assistant work assignments, creating duty rosters, assigning them to truck schedules, and updating delivery status; frequently used
- **Executive Management** (`ANALYST`): company executives requiring high-level operational and financial insight; read-only access to reporting, used infrequently
- **System Administrator** (`ADMIN`): manages staff accounts and roles and has access to all functions; used infrequently, high technical expertise
- **Database Administrator (DBA)**: maintains schema, indexes, stored procedures, and backups; used infrequently, high technical expertise
- **Customers (v2)** (`CUSTOMER`): wholesale and retail customers who use the customer portal to place orders and track their own orders and deliveries; moderate use, low technical expertise. Customers can never see other customers' data or any staff function.

## 2.4 Operating Environment

This system requires MySQL 8.0+ as the database server, running on Linux or Windows Server. The application server runs on JRE 17+, hosting a Spring Boot REST API application. The web interface is a React single-page application served by a web server (nginx). The three components can be deployed together with Docker Compose. Clients access the system through a modern web browser (Google Chrome, Firefox, or Microsoft Edge) over standard HTTP/HTTPS. Headquarters and destination store hubs are connected over a LAN/WAN using TCP/IP. No offline or native mobile operation is supported for Version 2.0.

## 2.5 Design and Implementation Constraints

- The system must use MySQL 8.0+ and a Spring Boot/JRE 17+ backend, as fixed implementation choices for this version
- ACID properties must be present at the database level, using stored procedures, functions, and triggers where appropriate (checking capacity requirements, roster conflicts, and route rules)
- Business rules must be enforced inside the database, so that they hold for every client and for direct data changes, not only for actions taken through the web interface **(v2)**
- Primary and foreign keys must be defined on all entities to maintain proper references to and from them
- Indexes must be applied on columns used for heavy filtering and reporting (order date, destination store, route identifier, trip date)
- The system must contain test data with at least 40 orders, 10 distinct routes, and valid train schedules with predefined capacities sufficient to exercise all test scenarios (detailed in Section 6)

## 2.6 User Documentation

A glossary (Appendix A) and a data dictionary (Appendix C) are provided with this SRS. As the current scope of the UI is basic, a separate user manual is not planned for Version 2.0; on-screen labels, tooltips, and the business-rule messages shown when an action is refused are considered sufficient.

## 2.7 Assumptions and Dependencies

- Each defined product type has a fixed space consumption rate (cargo units per item), backed by product data
- Each train schedule runs once a week on a fixed day, has a fixed total cargo capacity per run according to the Sri Lanka Railways schedule, and serves exactly one destination store
- The timetable used for testing has three trains and twelve weekly runs, two to each destination city, with cargo allocations of 100 to 200 space units per run. These figures are working assumptions until Sri Lanka Railways confirms them (see Appendix D)
- Routes are last-mile truck routes that start from a regional store. Each route covers a defined set of delivery areas, delivery areas do not overlap between routes, and each route has a defined maximum delivery time **(v2)**
- Each customer is registered in exactly one delivery area, and orders are delivered to the customer's registered address **(v2)**
- The duration of a truck trip is the scheduled time between dispatch and return. It must not exceed the maximum delivery time of the trip's route, and weekly driver/assistant hours are the sum of these durations within a Monday-to-Sunday week **(v2)**
- A truck trip starts and finishes on the same day
- There is one receiving store for each destination city unless specified otherwise

- Currency for reporting is Sri Lankan Rupees (LKR); quantities are tracked in boxes or units, consistent with each product's defined space consumption rate; route distances are in kilometres and delivery times in hours

# 3. External Interface Requirements

## 3.1 User Interfaces

The Kandypack integrated Logistics Platform provides a web-based graphical user interface (GUI) that enables different users to perform their tasks efficiently. Staff and customers sign in separately, and each user is given access according to their role after authentication.

The interface includes the following modules:

- **Staff Login Screen**
  - Secure authentication using username and password.
  - Redirects staff according to their assigned roles.
- **Customer Login and Registration (v2)**
  - A separate sign-in page for customers, linked from the staff login page.
  - Registration asks for name, email, phone, street address, delivery area (chosen from a list), username, and a password of at least 12 characters.
- **Dashboard**
  - Provides staff with a summary of current system activities, including pending orders, upcoming train trips, truck schedules, inventory levels, and recent deliveries.
- **Customer Portal**
  - Allows customers to view and update their profile and change their password.
  - Allows customers to place new orders by selecting products and quantities and a delivery date at least seven days ahead.
  - Shows customers their own order history with each order's status and delivery status. Customers cannot see other customers' orders or any staff function.
- **Order Management Interface**
  - Allows order clerks to place orders on a customer's behalf by selecting products and specifying quantities.
  - Supports orders containing multiple products with different quantities, and lists each customer's existing orders.
  - Validates that orders are placed at least seven days before the requested delivery date.
- **Train Schedule Interface**
  - Displays the train runs operating on a selected date, their destination cities, allocated cargo space, remaining capacity, and overflow schedules.
- **Store & Inventory Interface**

  - Shows products available at each destination store.
  - Updates inventory automatically after train arrival and truck dispatch.
- **Truck Dispatch Interface**
  - Allows roster coordinators to assign trucks, drivers, assistants, and delivery routes, and to move deliveries through their status steps.
  - Displays route duration and delivery status.
- **Reports Interface**
  - Enables management to generate quarterly sales reports, most ordered items, city-wise and route-wise sales, truck utilization reports, employee working hours, and customer order history, and to download each as a PDF.

The interface should remain responsive on modern desktop web browsers. When an action breaks a business rule, the interface shall show the rule's own message (for example, "Schedule Conflict: The Truck, Driver, or Assistant is already booked for an overlapping time window.") rather than a generic server error.

## 3.2 Hardware Interfaces

The system interacts with standard computer hardware available in the Kandypack head office and regional stores.

The hardware interfaces include:

| **Hardware** | **Purpose** |
|---|---|
| Desktop/Laptop Computers | Accessing the web application |
| Central Database Server | Stores all operational data |
| Network Router | Communication between headquarters and regional stores |

The application communicates with these devices through standard operating system drivers and TCP/IP networking.

## 3.3 Software Interfaces

The Kandypack Logistics Platform integrates with several software components to perform its operations.

| **Software Component** | **Interface** | **Purpose** |
|---|---|---|
| MySQL Database Server | JDBC | Stores all logistics data and executes SQL queries, stored procedures, and triggers |
| Backend Application (Spring Boot) | REST API (JSON over HTTP) | Processes business logic and communicates with the database |
| Web Interface (React) | HTTP/HTTPS | Provides the staff interface and the customer portal |

| Software Component | Interface | Purpose |
|---|---|---|
| Operating System (Windows/Linux) | System Services | File management, networking, and security |
| Reporting Module | Stored procedures / PDF generator | Generates management reports, displays them, and exports them as PDF |
| Docker Compose (v2) | Container runtime | Packages and starts the database, backend, and web interface together |

The backend application communicates with the relational database to store and retrieve logistics data, while the frontend exchanges data with the backend through standard web requests.

### 3.4 Communications Interfaces

The system communicates over a secure local or internet network using standard communication protocols.

The communication interfaces include:

- HTTP and HTTPS protocols for communication between the web browser and the application server.
- Session-based authentication using an HTTP-only session cookie, with a CSRF token required on every request that changes data **(v2)**.
- TCP/IP networking for communication between the headquarters and destination stores.
- Secure database connections between the backend server and the MySQL database, with database credentials supplied through environment variables rather than stored in source code.

## 4. System Features

### 4.1 Train Cargo Capacity & Space Consumption Scheduling

4.1.1 Description and Priority

- Description : Determines the utilization of train capacities based on product-specific space consumption, and schedules bulk shipments on the Sri Lanka Railways runs that go to each order's destination store. Manages overflow by scheduling overbooked items for the next run.
- Priority : High

4.1.2 Stimulus/Response Sequences

- User : Selects order lines and a target train run bound for the order's destination store.
- System : Sums the space required using the space rate per item (such as 0.5 space per box).
- System : Locks the train run and checks its remaining capacity on that date.

- System : Allocates capacity until the run is full and rolls the remaining quantity over to the next departure to the same store, first later that day, then on the following days on which a train to that store runs.
- System : If no capacity is found within four weeks, refuses the request with a clear message and saves nothing.

4.1.3 Functional Requirements

- REQ-1 : The system shall use the following formula to determine the amount of space being used: Total Space = Sum of Quantity_i * Space Consumption Rate_i.
- REQ-2 : Overbooking of trains shall not be allowed; a trigger shall check the remaining capacity before every shipment is saved.
- REQ-3 : When the volume of an order line exceeds the available cargo space, the system shall split it and send the excess to the next scheduled run to the same destination store.
- REQ-4 ***(v2)*** : Each train schedule shall reference its destination store by foreign key, and a shipment shall only be placed on a run whose destination store is the store of the order's route.
- REQ-5 ***(v2)*** : A shipment's date shall fall on the day of the week on which its train schedule runs.
- REQ-6 ***(v2)*** : Scheduling an order line shall be atomic (all of its shipments are saved, or none are), and concurrent scheduling shall not overbook a run.
- REQ-7 ***(v2)*** : If no capacity is available within four weeks, the system shall refuse the request with an error instead of searching indefinitely.
- REQ-8 ***(v2)*** : A train shall not be timetabled twice for the same day of the week and departure time.

## 4.2 Order Placement & Route Matching

4.2.1 Description and Priority

- Description : Enables customers (through the portal) and order clerks (on a customer's behalf) to order multiple items with at least 7 days advance notice. The system automatically assigns the delivery route that covers the customer's delivery area.
- Priority : High

4.2.2 Stimulus/Response Sequences

- User : Selects items, quantities, and the requested delivery date (no earlier than 7 days ahead).
- System : Checks the lead time (a minimum of 7 days), the products, and stock.
- System : Determines the route from the customer's delivery area.
- User : Approves and places the order.
- System : Saves the order with its requested delivery date and the status Placed, or refuses it with a message naming the problem.

4.2.3 Functional Requirements

- REQ-1 : An order shall be placed at least one week in advance: **RequestedDeliveryDate - OrderDate >= 7 days**, enforced by the order placement procedure and by a CHECK constraint on the stored order.
- REQ-2 : The system shall handle orders with multiple items, each referencing a product and quantity. Each line total shall equal quantity x unit price, and the order total shall equal the sum of its line totals.
- REQ-3 : Orders shall automatically be assigned the route of the customer's delivery area. There shall be no default route; an order for a customer without a delivery area shall be refused.
- REQ-4 ***(v2)*** : The requested delivery date shall be stored with each order.
- REQ-5 ***(v2)*** : Orders containing an unknown product, a non-positive quantity, or a quantity above available stock shall be refused with a specific message, and nothing shall be saved.
- REQ-6 ***(v2)*** : Customers placing orders through the portal shall only be able to place orders for themselves.

## 4.3 Last-Mile Delivery & Truck Scheduling

4.3.1 Description and Priority

- Description : Operates dispatch services for trucks taking goods from the destination stores at the railway stations to the customers' delivery addresses.
- Priority : High

4.3.2 Stimulus/Response Sequences

- User : Picks orders that have arrived at a destination store and schedules them for delivery.
- System : Displays available trucks at that store, the route, its maximum delivery time, and constraints.
- User : Schedules deliveries with a truck, route, driver, and assistant.
- System : Validates the trip, records it, provides updates of delivery status, and keeps a record of the expected completion times.

4.3.3 Functional Requirements

- REQ-1 : The system shall only allow a truck to be dispatched on routes that start from the store at which the truck is based. This shall be checked when a trip is created and when it is changed.
- REQ-2 : Each truck trip shall stay within the maximum delivery time of its route, and its return time shall be later than its dispatch time on the same day.
- REQ-3 : The system shall support the delivery status sequence Scheduled -> In Transit -> Delivered, with Cancelled allowed before delivery. Delivered and Cancelled are final, and In Transit cannot return to Scheduled. The order's status shall change in the same transaction.

- REQ-4 ***(v2)*** : Each order shall have at most one delivery. A delivery shall use a trip on the same route as the order, its date shall equal the trip date, and it shall be at least 7 days after the order date. These rules are implemented once, in the stored procedure sp_check_delivery, called by the BEFORE INSERT and BEFORE UPDATE triggers on Delivery.
- REQ-5 ***(v2)*** : Each delivery status change shall record the user who made it in the audit log.

## 4.4 Driver & Assistant Roster & Constraint Management

4.4.1 Description and Priority

- Description : Assigns drivers and assistants to truck schedules while strictly validating working-hour caps and consecutive assignment constraints. **Two trips are consecutive when the second starts less than 30 minutes after the first returns**; a gap of exactly 30 minutes counts as rest.
- Priority : High

4.4.2 Stimulus/Response Sequences

- User : Schedules or changes a truck trip with a driver and a driver assistant.
- System : Ensures that weekly working hours are within permissible limits for drivers (40 hrs) and assistants (60 hrs).
- System : Ensures the consecutive-trip rules (driver: no two consecutive trips; assistant: at most two consecutive trips).
- System : Confirms the assignment if valid, or blocks it with an error message naming the rule that was broken.

4.4.3 Functional Requirements

- REQ-1 : Every truck schedule shall have exactly one driver and exactly one driver assistant.
- REQ-2 : The system shall not allow a driver to be scheduled for two consecutive truck trips (at least 30 minutes of rest between trips).
- REQ-3 : The system shall not allow an assistant to be scheduled for more than two consecutive trips.
- REQ-4 : The system shall enforce a maximum of **40 scheduled hours per week** for drivers.
- REQ-5 : The system shall enforce a maximum of **60 scheduled hours per week** for assistants.
- REQ-6 : No two overlapping time slots can be assigned to the same truck, driver, or assistant.
- REQ-7 ***(v2)*** : REQ-2 to REQ-6 and Section 4.3.3 REQ-1 and REQ-2 shall be checked both when a trip is created and when an existing trip is changed. A changed trip shall not be compared against its own previous version.

- REQ-8 ***(v2)*** : Each rule shall be implemented once, as its own stored procedure (`sp_check_trip_route`, `sp_check_schedule_conflict`, `sp_check_driver_consecutive`, `sp_check_assistant_consecutive`, `sp_check_weekly_hours`), called by the BEFORE INSERT and BEFORE UPDATE triggers on `TruckTrip`.

## 4.5 Management Reporting & Analytics

4.5.1 Description and Priority

- Description : Prepares operating, financial and utilization reports for management decision making.
- Priority : Medium

4.5.2 Stimulus/Response Sequences

- Manager : Switches to the reports module and selects the report type and parameters (e.g. Quarter, Month and City).
- System : Runs the corresponding stored procedure.
- System : Outputs formatted tabular and/or graphic reports to the screen, with an option to download a PDF.

4.5.3 Functional Requirements

- REQ-1 : Quarterly Sales Report: generate a sales breakdown by value (LKR) and volume (units and space) for each quarter.
- REQ-2 : Top Items: report the most ordered products for a given quarter.
- REQ-3 : Geographic Sales: provide a city-wise and route-wise sales breakdown.
- REQ-4 : Working Hours: create weekly and period summaries of hours worked by drivers and assistants against their limits.
- REQ-5 : Fleet Usage: produce a monthly truck utilization analysis (trips, operating hours, and last-mile distance). Completed deliveries shall count only deliveries with the status Delivered.
- REQ-6 : Customer History: output the detailed order history of customers, including fulfilment and delivery details.
- REQ-7 ***(v2)*** : Each report shall be downloadable as a PDF.
- REQ-8 ***(v2)*** : Access to each report shall be limited by role: analysts and administrators see all reports, roster coordinators also see working hours, fleet managers also see fleet usage, and order clerks also see customer history.

## 4.6 Database Integrity & ACID Enforcement (System Internal Feature)

4.6.1 Description and Priority

- Description : System-level capability that enforces consistency, key relationships, triggers, and constraint checks in the database.
- Priority : High

4.6.2 Stimulus/Response Sequences

- System/User Action : User executes database transactions such as placing orders, assigning deliveries, and updating schedules.
- Database Engine : Runs constraints, foreign key checks, triggers, and transactions.
- System : Commits changes when ACID conditions are met, or rolls back changes on exception.

4.6.3 Functional Requirements

- REQ-1 : The system shall use explicit ACID-compliant transactions for train capacity scheduling, order placement, truck trip assignment, and delivery status updates.
- REQ-2 : Triggers shall run automatically to check the roster and route rules before a trip is inserted into or updated in `TruckTrip`. (Version 1.0 referred to a `RosterDetail` table; `TruckTrip` holds the roster.)
- REQ-3 : For reporting queries, indexes shall be added on frequently queried fields (e.g. `RouteID`, `OrderDate`, `DriverID`, `TripDate`) to improve the performance of such queries.
- REQ-4 ***(v2)*** : A business-rule violation shall be raised as SQLSTATE 45000 with a descriptive message, and the API shall return it to the user as HTTP 400 with that message.
- REQ-5 ***(v2)*** : Test data shall load with foreign key checks enabled, so that keys and triggers validate it, and an integrity test script shall confirm that it contains no rule violations.

# 5. Other Nonfunctional Requirements

## 5.1 Performance Requirements

- **Transactional Response Time**

  All standard daily operations, like entering new orders, checking delivery routes, and updating order status, must finish within 1.5 seconds during regular working hours.

- **Data Capacity and Scalability**

  The database should be designed and indexed properly so that it continues to run fast even after storing more than 100,000 order records and delivery details.

- **Report Generation Speed**

  Complex reports, such as quarterly sales summaries and monthly truck usage analysis, must be generated within 3 to 5 seconds even when processing a full year of sales data.

- **System Concurrency**

  The system must handle 30 to 50 users working at the same time, like store managers and dispatch staff, without slowing down or locking up the database. Concurrent bookings of the same train run must not overbook it.

## 5.2 Safety Requirements

- **Database Transaction Safety**

  The system must follow strict ACID properties for all operations, such as assigning orders to trains or trucks, so that data is never lost or partially saved if there is a sudden power loss or system failure.

- **Operational and Workload Safety**

  The database must automatically check and enforce maximum working hour and rest rules for drivers and assistants, whenever a trip is created or changed, to prevent fatigue and ensure safety during road deliveries.

- **Backup and Data Recovery**

  The system should create automated daily backups of the database to make sure no operational data or customer order history is lost in case of unexpected hardware damage. The backup tooling and retention period are listed in Appendix D.

## 5.3 Security Requirements

- **User Authentication and Role-Based Access**

  Users must log in with valid credentials. Staff and customers sign in through separate logins. The system must restrict access based on user roles so that normal staff cannot modify master data or override roster limits, and so that customers can only use the customer portal; any staff function requested by a customer must be refused.

- **Session Security and Timeout**

  The system must automatically log out inactive users after 15 minutes of idle time to prevent unauthorized access from unattended computers in offices or delivery hubs. The session identifier must change when a user logs in, and every request that changes data must carry a valid CSRF token.

- **Password and Credential Protection**

  Staff and customer passwords must be stored only as BCrypt hashes and must be between 12 and 72 characters long. No API response or screen may ever include a password or password hash. The source code and configuration files must not contain default passwords or database credentials; these are supplied through environment variables.

- **Customer Data Isolation (v2)**

  The customer whose data is read or changed must always be taken from the logged-in session, never from a value in the request. A customer asking for another customer's order must receive a "not found" response.

- **Database Action Auditing**

  Important system actions, such as changing order and delivery statuses and creating or modifying truck trips (including driver, assistant, truck, and time changes), must be recorded in an audit log with the username of the person who made the change and a timestamp.

## 5.4 Software Quality Attributes

- **System Reliability**

  The database must maintain continuous availability and strict transaction integrity so that order scheduling and cargo allocation never fail during peak business hours.

- **Database Maintainability**

  The relational structure must follow proper normalization standards (3NF) so that new tables, delivery routes, or features can be added in the future without breaking existing queries. For example, a customer's city is derived from their delivery area rather than stored a second time.

- **Operational Usability**

  The system interface should show clear error messages and guidance when a user violates business rules, such as trying to schedule an order with less than 7 days lead time.

- **Data Portability and Compatibility**

  The database schema must be designed using standard SQL so that it can be backed up, restored, or migrated across different relational database management systems without data loss.

## 5.5 Business Rules

- **Advance Order Placement Rule**

  Customers must place all orders **at least 7 days before the expected delivery date** so that train capacity and truck delivery routes can be planned properly. The requested delivery date is stored with the order.

- **Driver Workload and Roster Rule**

  A driver cannot be scheduled for two consecutive truck delivery trips (**at least 30 minutes of rest between trips**), and their total working time must **not exceed 40 hours in a single week**.

- **Assistant Workload and Roster Rule**

  A driver assistant can be assigned to a **maximum of two consecutive delivery trips** (trips less than 30 minutes apart), and their total working time must **not exceed 60 hours in a single week**.

- **Train Capacity and Cargo Overflow Rule**

  Fixed cargo capacity allocated by Sri Lanka Railways must be respected based on product space consumption rates. If an order exceeds the available train space, the excess quantity must automatically roll over to the next available run to the same destination store. Shipments may only use runs that go to the order's destination store, on the days those runs operate.

- **Schedule Conflict Prevention Rule**

  The system must prevent assigning the same truck, driver, or assistant to overlapping route times or conflicting delivery schedules, both when a trip is created and when it is changed.

- **Route and Store Rule (v2)**

  A truck may only serve routes that start from the store at which it is based, and a trip may not last longer than its route's maximum delivery time.

- **Delivery Area Rule (v2)**

  Every customer belongs to exactly one delivery area, and every delivery area is covered by exactly one route. The route of an order is always the route of the customer's delivery area.

# 6. Other Requirements

- **Relational Integrity and Normalization**

  The database schema must be organized using proper Primary Keys and Foreign Keys to maintain data consistency and prevent orphaned records across tables.

- **Database Stored Logic (Procedures, Functions, and Triggers)**

  Automated database triggers and procedures must be implemented to enforce business rules, such as checking driver weekly hours, preventing double-booking, checking route and store rules, and calculating train capacity before saving a transaction. Each rule is implemented once and reused by every caller.

- **Indexing Strategy**

  B-tree indexes must be applied on frequently queried columns, such as order date, route ID, delivery status, driver ID, train schedule ID, and trip date (including combined driver/date, assistant/date, and truck/date indexes for roster checks), to speed up search and report operations.

- **Test Data Provisioning**

  The database must be populated with initial test data that obeys every rule in this document, including: at least 40 customer orders spread over at least two quarters (April to September 2026) plus open orders awaiting delivery; all 12 delivery routes in use; a train timetable with at least two weekly departures to each of the six destination cities, including one city with two departures on the same day, and at least one order line that rolls over to a later run; and demonstration staff and customer accounts with properly hashed passwords.

- **Deployment (v2)**

  The whole system must start with a single Docker Compose command. The database is built from the deployment script on first start, and the backend must wait until the database is ready before starting.

# Appendix A: Glossary

- **FMCG (Fast-Moving Consumer Goods)**

  High-turnover consumer products that require fast, reliable transportation and scheduling across rail and road distribution channels.

- **Lead Time**

  The mandatory 7-day advance booking window required between when an order is placed and its requested delivery date.

- **Rollover Cargo**

  The portion of an order line's volume that exceeds a train run's allocated capacity limit and is automatically shifted to the next available run to the same destination store.

- **Consecutive Trips**

  Two trips for the same driver or assistant where the second starts less than 30 minutes after the first returns. A gap of exactly 30 minutes or more counts as rest.

- **Rest Gap (v2)**

  The minimum time, 30 minutes, between one trip's return and the next trip's dispatch for that trip not to count as consecutive.

- **Delivery Area (v2)**

  A named locality (for example, Nugegoda) covered by exactly one last-mile route. Every customer is registered in one delivery area.

- **Destination Store (v2)**

  The regional store at which a train run unloads its cargo, and from which last-mile routes start.

- **Requested Delivery Date (v2)**

  The date by which the customer wants an order delivered; it must be at least 7 days after the order date.

- **Rule Procedure (v2)**

  A stored procedure that checks one business rule for a truck trip and raises an error if the rule is broken. The trip triggers call each rule procedure in turn.

# Appendix B: Analysis Models

- **Entity Relationship Diagram (ERD)**

  Illustrates the normalized relational database design, showing primary key to foreign key mappings between customers, delivery areas, orders, train schedules, truck routes, employee rosters, and staff accounts. The Version 2.0 diagram is at the end of Appendix C.

- **Data Flow Diagram (DFD)**

  Shows the flow of information from customer order entry through train capacity allocation, rollover handling, truck routing, and driver roster validation.

- **Process Flowchart**

  Visualizes the step-by-step logic for validating driver weekly hour limits, checking consecutive trip rules, and assigning overflow cargo to subsequent train runs.

# Appendix C: Data Dictionary / Entity Tables

Constraints are listed in the Description column. FK = foreign key, PK = primary key, UQ = unique. Items added or changed in Version 2.0 are marked (v2).

### Customer

A person or business that places orders with Kandypack.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **CustomerID** | INT | PK | Unique identifier for the customer |
| FullName | VARCHAR(100) | | Customer's full name / business name |
| Email | VARCHAR(100) | UQ | Email address |
| ContactNumber | VARCHAR(15) | | Phone number |
| Address | VARCHAR(200) | | Street address used for delivery |
| AreaID | INT | FK | References DeliveryArea.AreaID; determines the delivery route. Replaces City (v2) |
| Username | VARCHAR(50) | UQ | Username used to log in to the customer portal |
| Password | VARCHAR(255) | | BCrypt hash of the customer's password; never returned by the API (v2) |

### DeliveryArea (v2)

A locality covered by exactly one last-mile route.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **AreaID** | INT | PK | Unique identifier for the delivery area |
| AreaName | VARCHAR(60) | | Name of the locality, e.g. Nugegoda |
| RouteID | INT | FK | References Route.RouteID; the route that covers this area |

*Constraint: UNIQUE (RouteID, AreaName).*

### Order

A customer order for one or more products. Stored in the table Orders, because ORDER is a reserved word in SQL.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **OrderID** | INT | PK | Unique identifier for the order |
| CustomerID | INT | FK | References Customer.CustomerID |
| RouteID | INT | FK | References Route.RouteID; assigned from the customer's delivery area |
| AdminID | INT | FK | References Administrator.AdminID; staff member who processed the order (optional) |

| Attribute | Type | Key | Description |
|---|---|---|---|
| OrderDate | DATE | | Date the order was placed |
| RequestedDeliveryDate | DATE | | Requested delivery date; at least 7 days after OrderDate (v2) |
| Status | ENUM | | Placed / Processing / Shipped / In Transit / Delivered / Cancelled |
| TotalAmount | DECIMAL(12,2) | | Total order value in LKR (sum of line totals) |

*Constraint: CHECK (DATEDIFF(RequestedDeliveryDate, OrderDate) >= 7) (v2).*

## OrderDetail

A single product line within an order (order-product junction).

| Attribute | Type | Key | Description |
|---|---|---|---|
| **OrderDetailID** | INT | PK | Unique identifier for the order line |
| OrderID | INT | FK | References Order.OrderID (deleted with its order) |
| ProductID | INT | FK | References Product.ProductID |
| Quantity | INT | | Units of the product ordered (> 0) |
| LineTotal | DECIMAL(12,2) | | Quantity x UnitPrice at the time of ordering |

## Product

An FMCG product that Kandypack distributes.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **ProductID** | INT | PK | Unique identifier for the product |
| ProductName | VARCHAR(100) | | Name of the product |
| Category | VARCHAR(50) | | Product category |
| UnitPrice | DECIMAL(12,2) | | Standard selling price in LKR (>= 0) |
| SpaceConsumption | DECIMAL(8,2) | | Train space used per unit, for capacity checks (> 0) |
| StockQuantity | INT | | Number of units currently in stock (>= 0) |

## Store

A regional distribution hub (Colombo, Negombo, Galle, Matara, Jaffna, Trincomalee) where goods arrive by rail before last-mile trucking.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **StoreID** | INT | PK | Unique identifier for the store |
| StoreName | VARCHAR(100) | | Name of the regional warehouse/store |
| City | VARCHAR(50) | | City where the store is located |

| Attribute | Type | Key | Description |
|---|---|---|---|
| Capacity | DECIMAL(10,2) | | Maximum storage capacity (> 0) |

### Route

A last-mile truck route that starts from a regional store and covers a set of delivery areas (v2: Version 1.0 described it as a rail route).

| Attribute | Type | Key | Description |
|---|---|---|---|
| **RouteID** | INT | PK | Unique identifier for the route |
| StoreID | INT | FK | References Store.StoreID; the store the route starts from |
| RouteName | VARCHAR(100) | | Name of the route |
| MaxDeliveryTime | DECIMAL(5,2) | | Maximum trip duration in hours (> 0) |
| Distance | DECIMAL(8,2) | | Estimated round-trip last-mile distance in kilometres (> 0) |

### Train

A train used to move goods from Kandy to the destination stores.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **TrainID** | INT | PK | Unique identifier for the train |
| TrainName | VARCHAR(100) | | Name/number of the train |
| MaxCapacity | DECIMAL(10,2) | | Maximum cargo capacity in space units (> 0) |

### TrainSchedule

A weekly run of a specific train from Kandy to one destination store.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **ScheduleID** | INT | PK | Unique identifier for the schedule entry |
| TrainID | INT | FK | References Train.TrainID |
| StoreID | INT | FK | References Store.StoreID; destination store. Replaces the Destination text column (v2) |
| CargoCapacity | DECIMAL(10,2) | | Space units allocated to Kandypack on each run (> 0) |
| DepartureTime | TIME | | Scheduled departure time from Kandy |
| ArrivalTime | TIME | | Scheduled arrival time |
| DayOfWeek | ENUM | | Day the schedule runs (Monday to Sunday) |

*Constraint: UNIQUE (TrainID, DayOfWeek, DepartureTime) (v2).*

### Shipment

The portion of an order line's goods carried on a particular train run.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **ShipmentID** | INT | PK | Unique identifier for the shipment |
| OrderDetailID | INT | FK | References OrderDetail.OrderDetailID (v2: Version 1.0 listed OrderID) |
| ScheduleID | INT | FK | References TrainSchedule.ScheduleID |
| Quantity | INT | | Units of the order line carried on this run (> 0) |
| ShipmentDate | DATE | | Date of the run; must fall on the schedule's DayOfWeek (v2) |

### Truck

A last-mile delivery truck based at a regional store.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **TruckID** | INT | PK | Unique identifier for the truck |
| RegistrationNumber | VARCHAR(20) | UQ | Vehicle registration number |
| Capacity | DECIMAL(10,2) | | Maximum cargo capacity (> 0) |
| StoreID | INT | FK | References Store.StoreID (home store) |

### Driver

A driver who operates delivery trucks.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **DriverID** | INT | PK | Unique identifier for the driver |
| Name | VARCHAR(100) | | Driver's full name |
| LicenceNumber | VARCHAR(30) | UQ | Driving licence number |
| ContactNumber | VARCHAR(15) | | Phone number |

### Assistant

An assistant who accompanies a driver on deliveries.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **AssistantID** | INT | PK | Unique identifier for the assistant |
| Name | VARCHAR(100) | | Assistant's full name |
| ContactNumber | VARCHAR(15) | | Phone number |

### TruckTrip

A scheduled truck dispatch: one truck, one route, one driver, and one assistant, for a given time window. Deliveries reference this trip rather than the truck/driver/assistant directly. Every insert and update is validated by the trip rule procedures (v2).

| Attribute | Type | Key | Description |
|---|---|---|---|
| **TripID** | INT | PK | Unique identifier for the truck trip |
| TruckID | INT | FK | References Truck.TruckID; must be based at the route's store (v2) |
| RouteID | INT | FK | References Route.RouteID |
| DriverID | INT | FK | References Driver.DriverID |
| AssistantID | INT | FK | References Assistant.AssistantID (mandatory) |
| TripDate | DATE | | The date on which the truck trip takes place |
| DispatchTime | TIME | | Scheduled dispatch time |
| ReturnTime | TIME | | Scheduled return time; later than DispatchTime, within the route's MaxDeliveryTime (v2) |

### Delivery

The last-mile delivery of an order from a store to the customer.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **DeliveryID** | INT | PK | Unique identifier for the delivery |
| OrderID | INT | FK, UQ | References Order.OrderID (one delivery per order) |
| TripID | INT | FK | References TruckTrip.TripID; trip must be on the order's route (v2) |
| DeliveryDate | DATE | | Date of delivery; equals the trip's TripDate (v2) |
| Status | ENUM | | Scheduled / In Transit / Delivered / Cancelled |

### Administrator (v2)

A staff user account. (Present in the database in Version 1.0 but not documented.)

| Attribute | Type | Key | Description |
|---|---|---|---|
| **AdminID** | INT | PK | Unique identifier for the staff account |
| Name | VARCHAR(100) | | Staff member's name |
| Username | VARCHAR(50) | UQ | Username used to log in |
| Password | VARCHAR(255) | | BCrypt hash of the password; never returned by the API |
| Email | VARCHAR(100) | UQ | Email address |

| Attribute | Type | Key | Description |
|---|---|---|---|
| Enabled | BOOLEAN | | Whether the account may log in |

### StaffRole (v2)

A role that grants a set of staff permissions.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **RoleID** | INT | PK | Unique identifier for the role |
| Code | VARCHAR(40) | UQ | ADMIN, ORDER_MANAGER, RAIL_DISPATCHER, FLEET_MANAGER, ROSTER_DISPATCHER, or ANALYST |
| DisplayName | VARCHAR(80) | | Name shown in the interface |

### AdministratorRole (v2)

Links staff accounts to their roles (many-to-many junction).

| Attribute | Type | Key | Description |
|---|---|---|---|
| **AdminID** | INT | PK, FK | References Administrator.AdminID (deleted with the account) |
| **RoleID** | INT | PK, FK | References StaffRole.RoleID |

### AuditLog (v2)

A record of an important change to any table (Section 5.3). It has no foreign keys so that history is kept even if a record is later removed.

| Attribute | Type | Key | Description |
|---|---|---|---|
| **LogID** | INT | PK | Unique identifier for the log entry |
| TableName | VARCHAR(50) | | Table that was changed |
| ActionType | VARCHAR(20) | | INSERT, UPDATE, or DELETE |
| RecordID | INT | | Primary key of the changed record |
| ChangedBy | VARCHAR(50) | | Username of the person who made the change, or SYSTEM |
| ChangeTimestamp | TIMESTAMP | | When the change was made (defaults to now) |
| Details | TEXT | | Description of the change, e.g. old and new status |

## Entity Relationships

The table below summarises the relationships between entities and their cardinality.

| Entity A | Entity B | Cardinality | Description |
|---|---|---|---|
| Customer | Order | 1 : M | A customer places many orders |

| Entity A | Entity B | Cardinality | Description |
|---|---|---|---|
| DeliveryArea | Customer | 1 : M | A delivery area contains many customers (v2) |
| Route | DeliveryArea | 1 : M | A route covers many delivery areas (v2) |
| Store | Route | 1 : M | A store is the starting point of many last-mile routes |
| Route | Order | 1 : M | A route serves many orders |
| Order | OrderDetail | 1 : M | An order contains many order lines |
| Product | OrderDetail | 1 : M | A product appears on many order lines |
| OrderDetail | Shipment | 1 : M | An order line may be split across train runs (v2: was Order - Shipment) |
| TrainSchedule | Shipment | 1 : M | A scheduled run carries many shipments |
| Train | TrainSchedule | 1 : M | A train has many scheduled runs |
| Store | TrainSchedule | 1 : M | A store is the destination of many runs (v2: was Route - TrainSchedule) |
| Store | Truck | 1 : M | A store has many trucks based there |
| Order | Delivery | 1 : 1 | An order results in one last-mile delivery |
| TruckTrip | Delivery | 1 : M | A truck trip may serve multiple deliveries |
| Truck | TruckTrip | 1 : M | A truck performs many trips |
| Driver | TruckTrip | 1 : M | A driver drives many trips |
| Assistant | TruckTrip | 1 : M | An assistant supports many trips |
| Route | TruckTrip | 1 : M | A route has many scheduled truck trips |
| Administrator | Order | 1 : M | A staff member may process many orders (optional) (v2) |
| Administrator | StaffRole | M : N | Staff accounts have roles, via AdministratorRole (v2) |

## Database Program Units (v2)

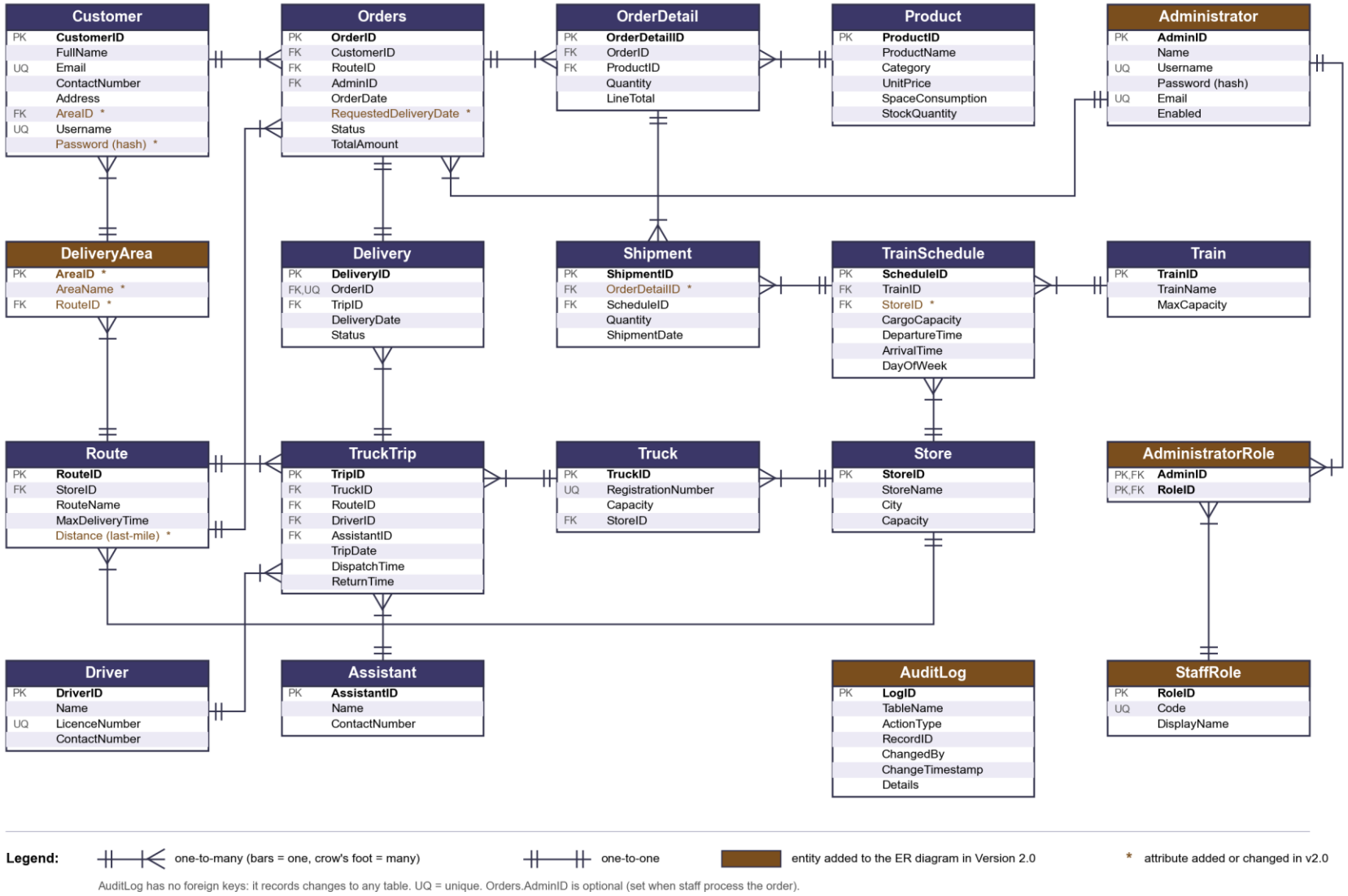
The stored logic that enforces the requirements in Sections 4 and 5.

| Name | Type | Purpose |
|---|---|---|
| fn_calculate_total_space | Function | Space needed for a quantity of a product (4.1.3 REQ-1) |
| fn_get_available_capacity | Function | Remaining cargo space on a train run on a date |
| fn_calculate_weekly_hours | Function | Hours a driver or assistant is scheduled in a week (used by views and reports through GetDriverWeeklyHours and GetAssistantWeeklyHours) |

| Name | Type | Purpose |
|---|---|---|
| fn_get_route_for_address | Function | Route of a customer's delivery area (4.2.3 REQ-3) |
| sp_place_order | Procedure | Validates and saves an order and its lines in one transaction |
| sp_schedule_shipment | Procedure | Books an order line onto train runs with rollover, atomically |
| sp_assign_truck_trip | Procedure | Saves a truck trip and its audit entry in one transaction |
| sp_update_delivery_status | Procedure | Changes delivery status and the order status together |
| sp_check_trip_route | Procedure (rule) | Return after dispatch; truck from the route's store; within maximum delivery time |
| sp_check_schedule_conflict | Procedure (rule) | No overlapping truck, driver, or assistant bookings |
| sp_check_driver_consecutive | Procedure (rule) | Driver has at least 30 minutes of rest between trips |
| sp_check_assistant_consecutive | Procedure (rule) | Assistant works at most two consecutive trips |
| sp_check_weekly_hours | Procedure (rule) | Driver 40 hours and assistant 60 hours per week |
| sp_check_delivery | Procedure (rule) | Delivery uses a trip on the order's route, on the trip's date, at least 7 days after the order date |
| Report procedures | Procedure | sp_quarterly_sales_report, sp_top_items_report, sp_geographic_sales_report, sp_working_hours_report, sp_staff_weekly_hours_report, sp_fleet_usage_report, sp_customer_order_history |
| trg_check_cargo_capacity | Trigger | Prevents train overbooking: refuses a shipment that needs more space than the run has left |
| trg_shipment_weekday_check_insert, trg_shipment_weekday_check_update | Trigger | Refuse a shipment dated on a day of the week on which its train schedule does not run |
| trg_update_stock_on_dispatch | Trigger | Reduces stock when cargo is dispatched |
| trg_before_insert_trucktrip, trg_before_update_trucktrip | Trigger | Run the five trip rule procedures on every new or changed trip |

| Name | Type | Purpose |
|---|---|---|
| trg_before_insert_delivery, trg_before_update_delivery | Trigger | Run sp_check_delivery on every new or moved delivery |
| trg_audit_order_status, trg_audit_delivery_status, trg_audit_trucktrip_update | Trigger | Write audit log entries for status and assignment changes |
| vw_order_summary, vw_truck_utilization, vw_staff_weekly_hours | View | Order summaries, truck usage, and current-week staff hours |

# ER Diagram


Customer
PK CustomerID
FullName
UQ Email
ContactNumber
Address
FK AreaID *
UQ Username
Password (hash) *
Orders
PK OrderID
FK CustomerID
FK RouteID
FK AdminID
OrderDate
RequestedDeliveryDate *
Status
TotalAmount
OrderDetail
PK OrderDetailID
FK OrderID
FK ProductID
Quantity
LineTotal
Product
PK ProductID
ProductName
Category
UnitPrice
SpaceConsumption
StockQuantity
Administrator
PK AdminID
Name
UQ Username
Password (hash)
UQ Email
Enabled
DeliveryArea
PK AreaID *
AreaName *
FK RouteID *
Delivery
PK DeliveryID
FK,UQ OrderID
FK TripID
DeliveryDate
Status
Shipment
PK ShipmentID
FK OrderDetailID *
FK ScheduleID
Quantity
ShipmentDate
TrainSchedule
PK ScheduleID
FK TrainID
FK StoreID *
CargoCapacity
DepartureTime
ArrivalTime
DayOfWeek
Train
PK TrainID
TrainName
MaxCapacity
Route
PK RouteID
FK StoreID
RouteName
MaxDeliveryTime
Distance (last-mile) *
TruckTrip
PK TripID
FK TruckID
FK RouteID
FK DriverID
FK AssistantID
TripDate
DispatchTime
ReturnTime
Truck
PK TruckID
UQ RegistrationNumber
Capacity
FK StoreID
Store
PK StoreID
StoreName
City
Capacity
AdministratorRole
PK,FK AdminID
PK,FK RoleID
Driver
PK DriverID
Name
UQ LicenceNumber
ContactNumber
Assistant
PK AssistantID
Name
ContactNumber
AuditLog
PK LogID
TableName
ActionType
RecordID
ChangedBy
ChangeTimestamp
Details
StaffRole
PK RoleID
UQ Code
DisplayName
Legend:
one-to-many (bars = one, crow's foot = many)
one-to-one
entity added to the ER diagram in Version 2.0
* attribute added or changed in v2.0
AuditLog has no foreign keys: it records changes to any table. UQ = unique. Orders.AdminID is optional (set when staff process the order).

# Appendix D: To Be Determined List

- **Rail Capacity Threshold Specifications**

  Exact volume and weight limits per train compartment provided by Sri Lanka Railways need formal confirmation to set the precise baseline for cargo overflow triggers.

- **Emergency Overtime Overrides**

  The specific administrative process and authorization role required to override the 40-hour driver limit during severe operational delays or emergencies need final approval.

- **System Database Host Selection**

  The exact database server hardware, cloud provider, and operating system specifications for hosting the production database remain to be finalized by the infrastructure team.

- **External ERP Integration Protocol**

  The exact API format and data transmission frequency for syncing order data with existing legacy accounting tools need to be defined.

- **Driver / Assistant System Access**

  Whether drivers and assistants require individual system logins, or remain data-only subjects tracked for rostering purposes, needs confirmation (see Sections 2.3 and 2.7).

- **Backup Tooling and Retention (v2)**

  The tool used for the automated daily backups required by Section 5.2 (for example, a scheduled database dump or a managed backup service), where backups are stored, and how long they are kept remain to be decided.

# Appendix E: Changes from Version 1.0

This appendix lists every change made to the Version 1.0 SRS of 25 July 2026. Rows marked Version 2.1 were added on 11 October 2026 to align the document with the system as built and tested.

| Section | Change | Reason |
|---|---|---|
| 1.1, 1.2 | Version updated; database-object example `RosterDetail` replaced; (v2) marker introduced | `RosterDetail` does not exist; the roster is held in `TruckTrip` |
| 1.4, 2.1, 2.2 | Customer portal and delivery areas added to the scope and product functions | Customers place and track their own orders; routes are matched by area |
| 2.3 | Customers added as a user class; system role names added | The portal introduces a customer user class |
| 2.4, 3.3, 6 | React frontend and Docker Compose deployment described; software interfaces table corrected | Reflects how the system is built and deployed; the Version 1.0 table had misaligned columns |

| Section | Change | Reason |
|---|---|---|
| 2.7 | Routes defined as last-mile truck routes with delivery areas; trip duration assumption changed | Version 1.0 derived trip duration from the route's maximum; trips now have scheduled times that must stay within it |
| 3.1, 3.4 | Customer portal and separate customer login specified; rule messages shown to users; CSRF protection added | Keeps the customer interface while protecting it |
| 4.1 | REQ-3 clarified; REQ-4 to REQ-8 added | Destination store as a foreign key, weekday matching, atomic scheduling with locking, a 4-week limit, and a unique timetable slot |
| 4.2 | Requested delivery date stored and checked; routes assigned from the delivery area; REQ-4 to REQ-6 added | Lead time can now be verified for every stored order; city-only matching never chose a city's second route |
| 4.3 | Truck-store and maximum-time rules apply on create and edit; delivery rules and status sequence specified | Wrong-store trucks, over-long trips, and mismatched deliveries were previously accepted |
| 4.4 | Consecutive trips defined as less than 30 minutes apart; rules apply on edit; one procedure per rule | Driver and assistant rules used different definitions, and edits skipped every rule |
| 4.5 | PDF export and role-based report access added | Reflects the reports module as built |
| 4.6 | REQ-2 corrected to `TruckTrip`; error handling (HTTP 400) and test data validation added | Users should see the rule that was broken; test data must obey the rules |
| 5.1, 5.2 | Concurrency and rest rules extended | Concurrent train bookings and trip edits must also be safe |
| 5.3 | Separate customer login, BCrypt for all passwords, no passwords in API responses, session ID renewal, no credentials in source code, customer data isolation, trip audit | Customer passwords were stored and returned unhashed; default credentials were in the source code |
| 5.4, App. C | `Customer.City` replaced by `AreaID` | Removes a value that can be derived (3NF) |
| 5.5 | Route and Store rule and Delivery Area rule added; consecutive-trip limits quantified | Makes all enforced rules explicit |
| App. A | Consecutive Trips redefined; five terms added | Matches Section 4.4 |
| App. C | Misaligned tables corrected; DeliveryArea, Administrator, StaffRole, AdministratorRole, and AuditLog documented; Shipment linked to OrderDetail; TrainSchedule linked to Store; | Version 1.0 tables had types and keys in the wrong columns and omitted tables that exist in the database |

| Section | Change | Reason |
|---|---|---|
| | program units table and new ER diagram added | |
| App. D | Backup tooling and retention added | Section 5.2 requires backups but the mechanism is not yet chosen |
| 1.5 (Version 2.1) | Module technical specifications added as a reference | Detailed as-built notes for the rail, roster and delivery, and order and portal modules are kept with the source code |
| 2.7, 4.5, App. C (Version 2.1) | Test timetable stated as an assumption; route distance defined as round trip; completed deliveries defined as deliveries with the status Delivered | Matches the seeded data and the fleet reports as built |
| 4.3, App. C (Version 2.1) | sp_check_delivery and the shipment weekday triggers named; the delivery triggers described as calling the rule procedure | The program units table omitted objects that exist in the database |